CircuitBuilder: From Polynomials to Circuits via Reinforcement Learning

Weikun K. Zhang Rohan Pandey Bhaumik Mehta Kaijie Jin Naomi Morato Archit Ganapule
Michael R. Zeng Jarod Alper
University of Washington

Introduction

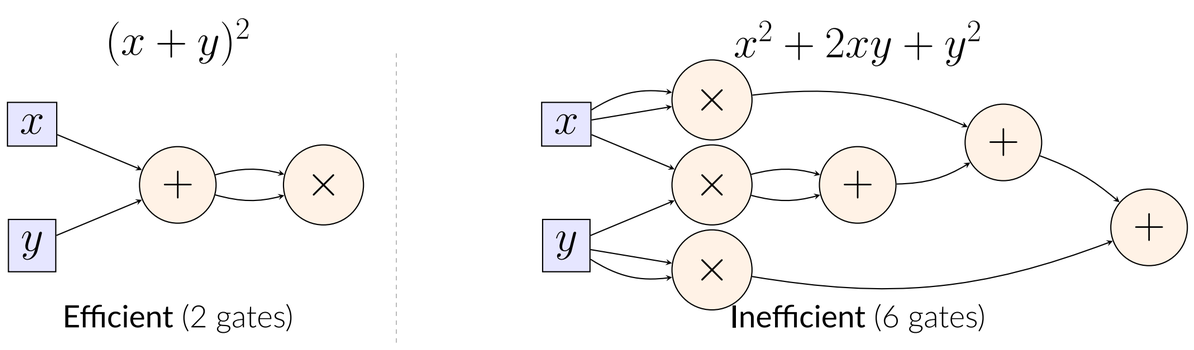
Arithmetic circuits compute polynomials using a sequence of $+$ and $\times$ gates applied to input variables. Determining the *smallest* such circuit for a given polynomial is a fundamental open problem in algebraic complexity theory, with deep connections to algorithm design and the **VP vs. VNP conjecture**—the algebraic analogue of P vs. NP. Deep learning often provides efficient solutions to NP and VNP hard problems. In this project, we train an RL agent to efficiently solve the arithmetic circuit problem.

- **Goal.** Given $f(x_1, \ldots, x_n)$, find a circuit using the minimum number of $+$ and $\times$ gates to compute it exactly.
- **Analogy.** Just as a proof is a sequence of deductions from axioms, a circuit is a sequence of operations from input variables— circuit synthesis directly mirrors proof search.
- **Challenge.** With k intermediate polynomials computed, there are $O(k^2)$ possible next operations, leading to exponential blowup in the search space.

Problem Formulation

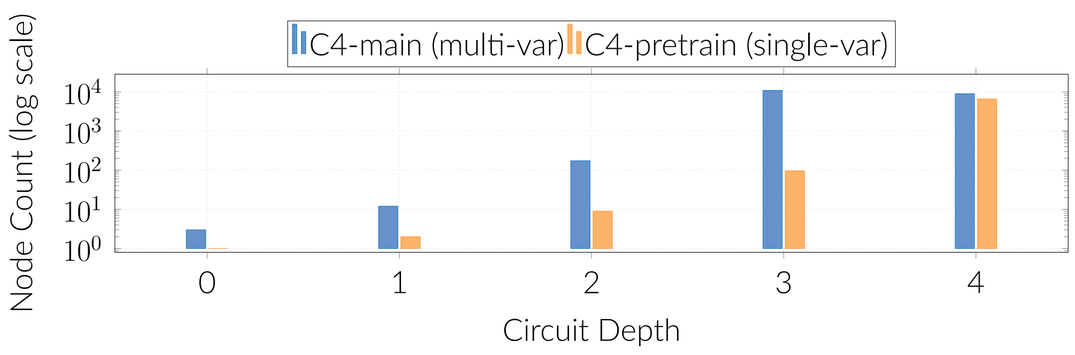
We model circuit construction as a **single-player MDP**. The problem can be viewed as a simplified version of auto-proof generation, where an agent applies a sequence of lemmas to prove a mathematical theorem.

- **State.** Computed set $\{p_1, \ldots, p_k\}$, seeded with $x_1, \ldots, x_n$ and 1.
- **Action.** Pick p_i, p_j and $\mathrm{op} \in \{+, \times\}$; append $p_{k+1} = p_i \ \mathrm{op} \ p_j$.
- **Termination.** Win when $p_k = f$; fail after N steps.



Gameboard Structure

The **C4 gameboard** is a precomputed DAG of reachable polynomials through depth 4; 94–99% of nodes admit multiple optimal circuits, confirming the vastness of the search space.



Architectural Approaches

Both agents share a **GNN-Transformer backbone**: GCNConv layers encode the circuit DAG, fused with a target polynomial embedding to produce policy logits and a value estimate.

- **SAC (off-policy).** Twin Q-heads; entropy bonus for sustained exploration; replay buffer with Polyak target updates. Complexity *decreases* if success rate falls below a lower threshold, preventing stalling.
- **CircuitBuilder (PPO+MCTS).** PPO trains the network; MCTS (64 simulations) deepens search via $\mathrm{UCB}(s,a) = Q(s,a)/N(s,a) + C\sqrt{\ln N(s)/N(s,a)}$. With p=0.35, MCTS overrides policy sampling during rollout, teaching the policy to match deeper look-ahead.

Reward Design

- **Success bonus.** $+100$ for computing the exact target.
- **Out-degree bonus.** Reward for reusing computed polynomials.

Factor Library & Reward Shaping. The *factor library* dynamically shapes rewards and introduces subgoals.

- **Divisibility.** If $p \mid g$, grant a reward and set the new subgoal to $q = g/p$.
- **Factorable residual.** If $g - p = \prod_i f_i$ factors non-trivially, grant a reward and add each f_i to the active subgoal set.

Target: $g = x^2 + 3x + 2 = (x+1)(x+2)$ • Agent computes: $p = x + 1$

↓ Two algebraic checks on p vs. g

Case 1: Divisibility
$p \mid g$ Reward ✓
New subgoal:
$g/p = x + 2$

Case 2: Residual
$g - p = (x+1)^2$ Reward ✓
New subgoals:
$\{x+1,\ x+1\}$

Key insight: Algebraic decomposition converts a sparse terminal signal into a structured sequence of tractable subgoals. This is analogous to auto-proof generation, where an agent breaks down a goal into a sequence of subgoals.

Training Pipeline

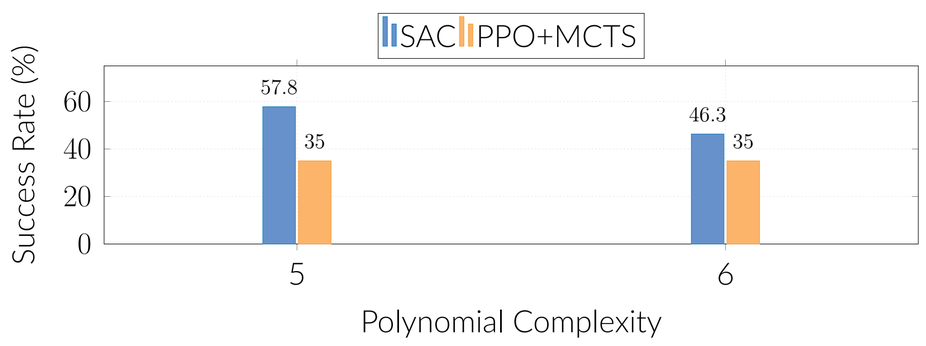
Two-phase curriculum:

1. **Supervised pretraining.** Cross-entropy (action) + MSE (value) on exhaustively-searched optimal circuits.
2. **RL fine-tuning (PPO or SAC).** GAE [Sch+16] (γ=0.99, λ=0.95); clipped objective (ϵ=0.2); entropy bonus (c_{ent}=0.02). Advance complexity when success rate $> 60\%$.

Hyperparameters: LR $= 10^{-5}$; batch 4096; 10 PPO epochs/iter; 2000 total iterations. Trained on AWS EC2 + NVIDIA Tesla T4 (16 GB).

Results & Conclusion

Success rates on held-out polynomials ($n = 2$ variables):



- **SAC** leads on two-variable targets, learning effective symbolic construction strategies.
- **PPO+MCTS** scales better to three variables; explicit planning becomes increasingly important with dimensionality.
- **Curriculum** is essential for higher complexity; **sparse rewards** remain the primary bottleneck.

Future work: Scale to degree-8+ polynomials; stronger exploration strategies; AlphaTensor-style [Faw+22] integration.

Acknowledgements: AWS credits (UW eScience / UW IT); UW Research Computing Club (Student Technology Fee); UWIT Tillicum GPU cluster.

References

[Bro+12] C. B. Browne et al. "A Survey of Monte Carlo Tree Search Methods". In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (2012), pp. 1–43.
[Faw+22] A. Fawzi et al. "Discovering faster matrix multiplication algorithms with reinforcement learning". English. In: *Nature, London* 610.7930 (2022), pp. 47–53. DOI: 10.1038/s41586-022-05172-4.
[Hub+25] T. Hubert et al. "Olympiad-level formal mathematical reasoning with reinforcement learning". In: *Nature* (2025). DOI: 10.1038/s41586-025-09833-y.
[KW17] T. N. Kipf and M. Welling. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations (ICLR)*. 2017.
[Sch+16] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". In: *International Conference on Learning Representations (ICLR)*. 2016.
[Sch+17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).
[Sil+17] D. Silver et al. "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm". In: *arXiv preprint arXiv:1712.01815* (2017).
[Vas+17] A. Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.